课程网站

- https://kyon-hs.github.io/JLU-COA/
  - slides等

邀请码: 48622429

APP首页右上角输入



该邀请码2027年03月06日前有效

王班

邀请码: 39303259

APP首页右上角输入



该邀请码2027年03月06日前有效

其他班级

评价标准

- **平时20%+作业20%+期末60%**
  - 4次作业，各5%，晚于Date Due提交会失去成绩
  - 前两次每次5道选择题，第三次5道判断题，第四次2道大题类型的选择题
  - **期末，王湘浩实验班与其他班试卷会有区别**
  - **期末一般是10-15道选择题，10-15道判断题，2-3道大题**

- **五个宽限日**
  - 作业或者签到
  - 用完后，我们不会接受任何延长时限的请求
  - 例如：Date Due 5月13日，如使用两个宽限日，5月15日23:59前提交都可以

指令集架构的设计权衡

今天的内容

- 指令集架构的设计权衡

上节课回顾

- 转换层次
  - 算法、ISA（指令集体系结构）、微架构
- 基本概念
- 冯·诺依曼模型
- 数据流模型
- ISA 与微架构的比较

回顾：指令集架构（ISA）



- 指令（Instructions）
  - 操作码、寻址方式、数据类型
  - 指令类型与格式（Instruction Types and Formats）
  - 寄存器、条件码（Registers, Condition Codes）
- 内存（Memory）
  - 地址空间、可寻址性、对齐
  - 虚拟内存管理（Virtual memory management）

Intel® 64 and IA-32 Architectures
Software Developer's Manual

Volume 1:
Basic Architecture

- 其他内容
  - 调用、异常/中断处理（Call, Interrupt/Exception Handling）
  - 访问控制、优先级/特权（Access Control, Priority/Privilege）
  - 输入输出（I/O）：内存映射 I/O vs. 指令驱动 I/O
  - 任务/线程管理（Task/thread Management）
  - 功耗与热管理（Power and Thermal Management）
  - 多线程与多处理器支持（Multi-threading and Multiprocessor support）

回顾：微架构（Microarchitecture）

- 在特定**设计约束与目标（design constraints and goals）**下对 ISA 的具体实现。
- 任何在硬件层面完成且不暴露给软件的设计内容都属于微架构范畴。
  - 流水线（Pipelining）
  - 顺序执行 vs. 乱序执行（In-order vs. Out-of-order Execution）
  - 内存访问调度策略（Memory Access Scheduling Policy）
  - 推测执行（Speculative Execution）
  - 超标量处理（Superscalar Processing，多指令并发）
  - 时钟门控（Clock Gating）
  - 缓存设计（Caching：层级、容量、关联度、替换策略）
  - 预取机制（Prefetching）
  - 电压/频率动态调节（Voltage/Frequency Scaling）
  - 错误校正（Error Correction）

ISA 与微架构（Uarch）的特性区别

- 加法（ADD）指令的操作码
- 通用寄存器（general purpose registers）的数量
- 寄存器文件（register file）的端口数
- 执行乘法（MUL）指令所需的时钟周期数
- 机器是否采用流水线（pipelined）指令执行

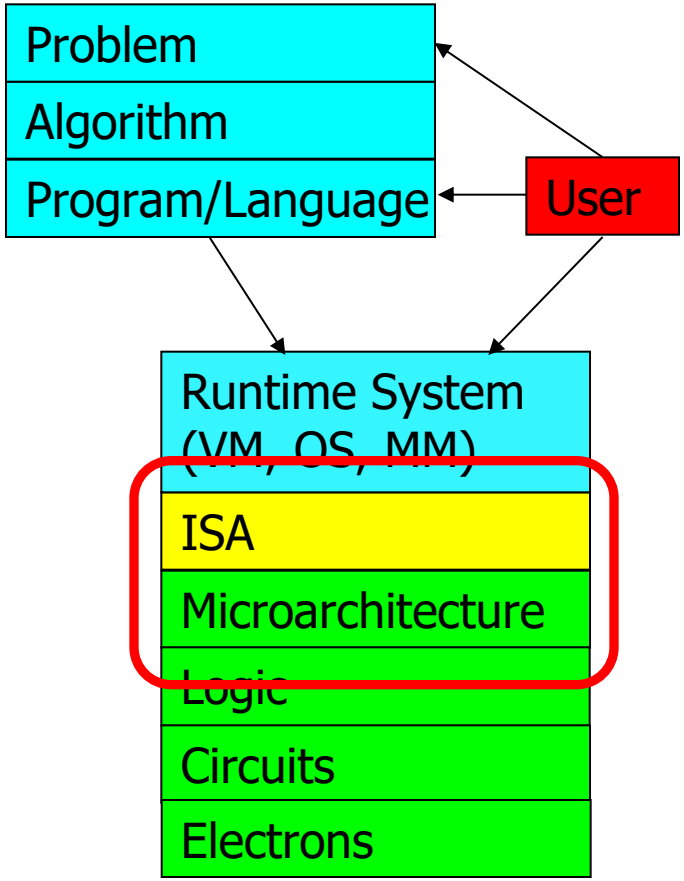
- 请记住：
  - 微架构（Microarchitecture）：是在特定设计约束和目标下对 ISA（指令集体系结构） 的实现。

权衡：计算机体系结构的灵魂

- ISA 层面的权衡
- 微架构层面的权衡
- 系统与任务层面的权衡
  - 如何在硬件与软件之间划分工作
- 计算机体系结构是一门科学与艺术，旨在通过恰当的权衡来实现设计目标。
  - 为什么说是“艺术”？

为什么被称为“艺术”？

来自上层的新需求
（向上看）



来自用户的新需求
与个性（向上看）

来自底层的新问题
与新能力（向下看）

- 我们并不（完全）了解未来（应用、用户、市场）。
- 而且，未来并不是固定不变的（它会变化）！

来自宏观建筑的类比

- 在宏观建筑中，未来同样不是一成不变的。

- 例如：锅炉房能否在之后被改造成教室？

我们如何适应未来

- 这是一名优秀计算机体系结构师的重要职责之一。

我们如何适应未来

- 这是一名优秀计算机体系结构师的重要职责之一。

- 许多可行的方法
  - 敏锐的洞察力与良好的设计
  - 善用基本原理与设计原则
    - 高效设计
    - 异构性（多样化体系结构）
    - 可重构性（可根据需求调整架构）
    - ……
  - 充分利用底层技术
  - ……

指令集架构的设计权衡

数十年来的多种指令集架构（ISAs）

- x86
- PDP-x（Programmed Data Processor，如 PDP-11）
- VAX
- IBM 360
- CDC 6600
- SIMD 类型 ISA：如 CRAY-1、Connection Machine
- VLIW 类型 ISA：如 Multiflow、Cydrome、IA-64（EPIC）
- PowerPC / POWER
- RISC 类型 ISA：如 Alpha、MIPS、SPARC、ARM
- 思考：这些 ISA 的根本区别是什么？
  - 指令是如何定义和执行的？（how instructions are specified and what they do）
  - 指令的复杂度有多高？（how complex are the instructions）

指令（Instruction）

- 是硬件与软件接口（HW/SW interface）的基本元素。

- 组成部分（Consists of）
  - Opcode（操作码）：描述指令要执行的操作（what the instruction does）。
  - Operands（操作数）：指明操作对象（who it is to do it to）。

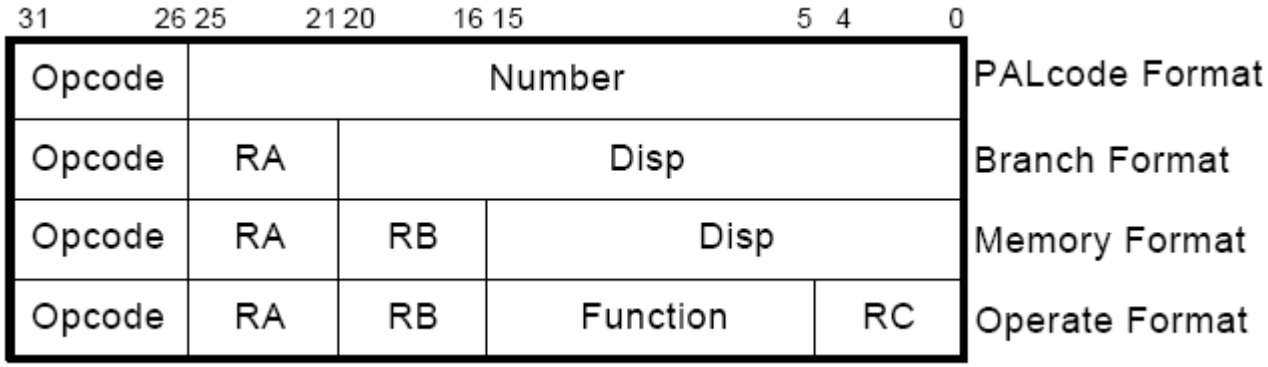
- 示例：Alpha ISA 中的指令格式

指令（Instruction）

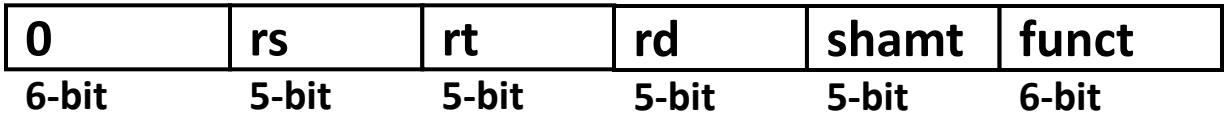
- 是硬件与软件接口（HW/SW interface）的基本元素。

- 组成部分（Consists of）
  - Opcode（操作码）：描述指令要执行的操作（what the instruction does）。
  - Operands（操作数）：指明操作对象（who it is to do it to）。

- 示例：Alpha ISA 中的指令格式

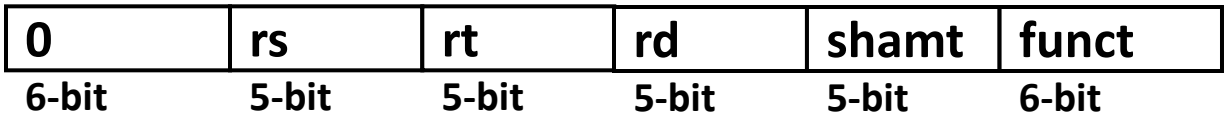


MIPS

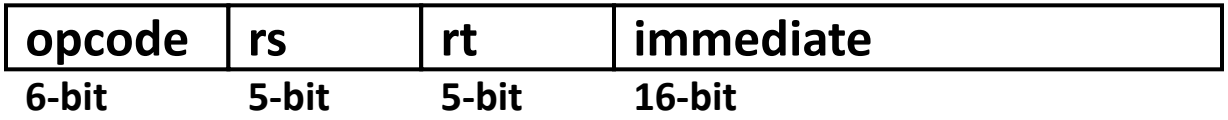


R-type

MIPS

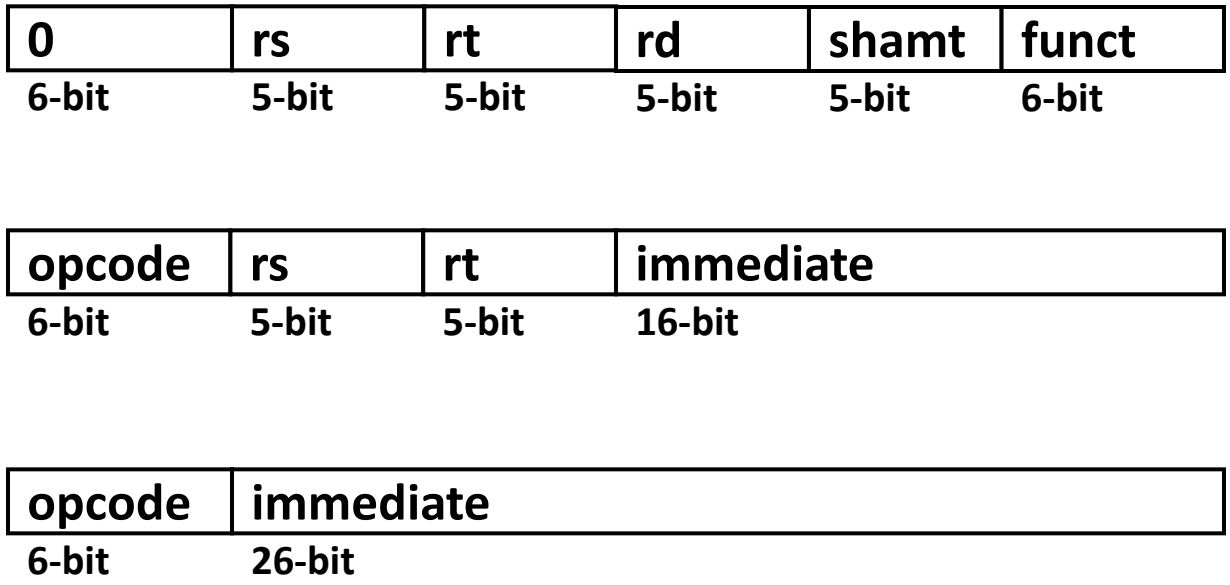


R-type



I-type

MIPS


0
rs
rt
rd
shamt
funct
6-bit
5-bit
5-bit
5-bit
5-bit
6-bit
R-type
opcode
rs
rt
immediate
6-bit
5-bit
5-bit
16-bit
I-type
opcode
immediate
6-bit
26-bit
J-type

ARM

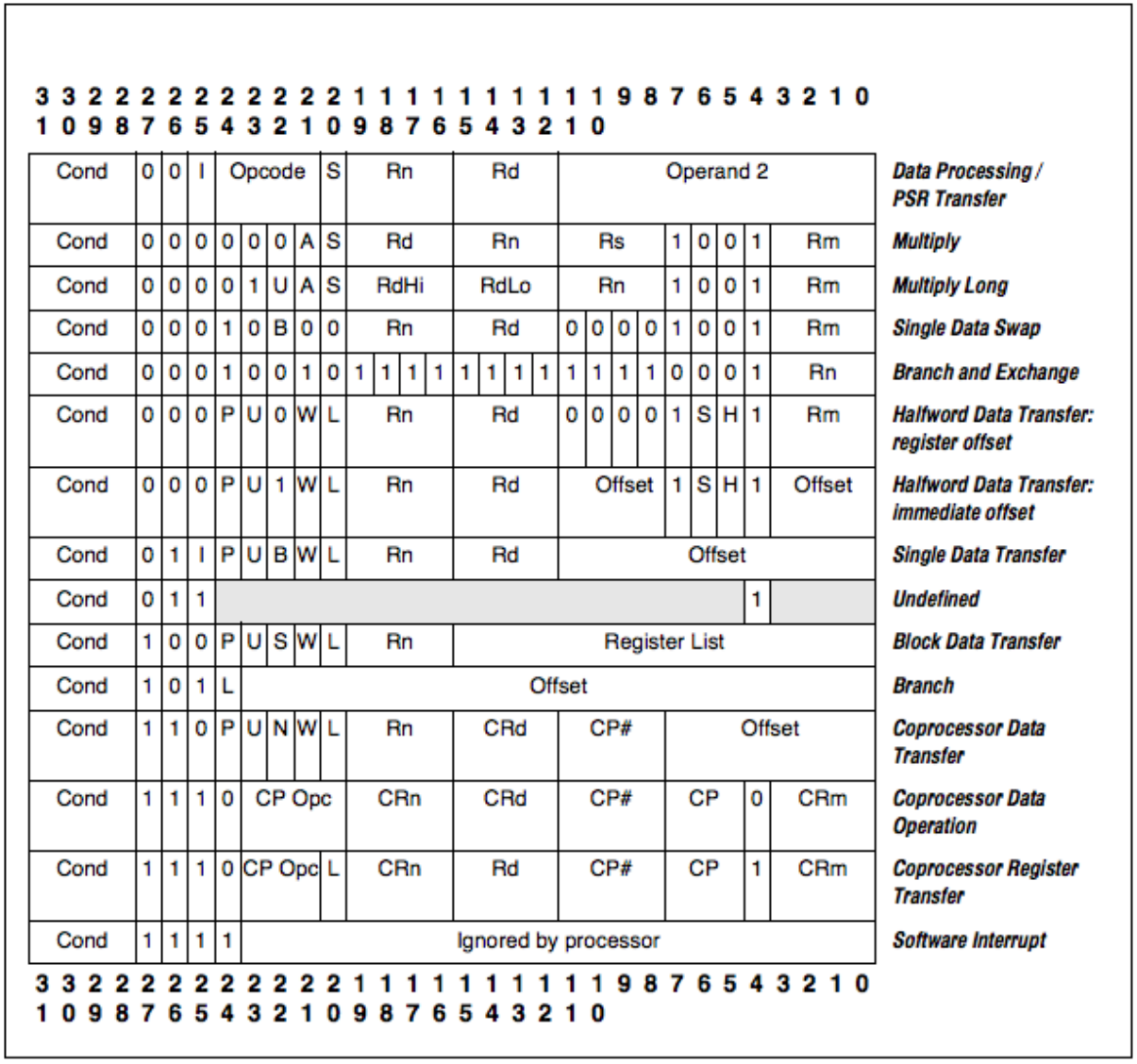


Figure 4-1: ARM instruction set formats

ARM

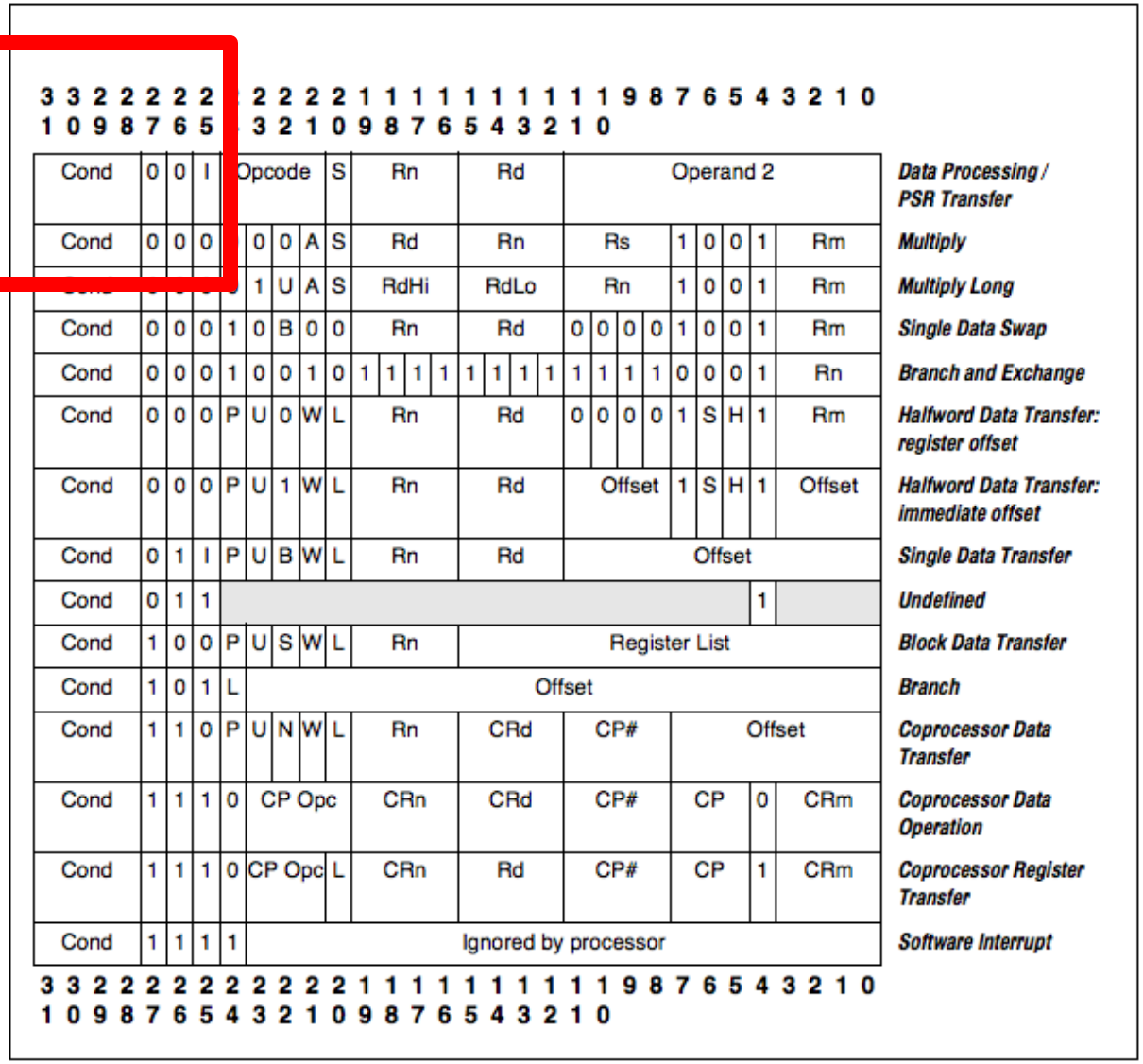


Figure 4-1: ARM instruction set formats

ARM

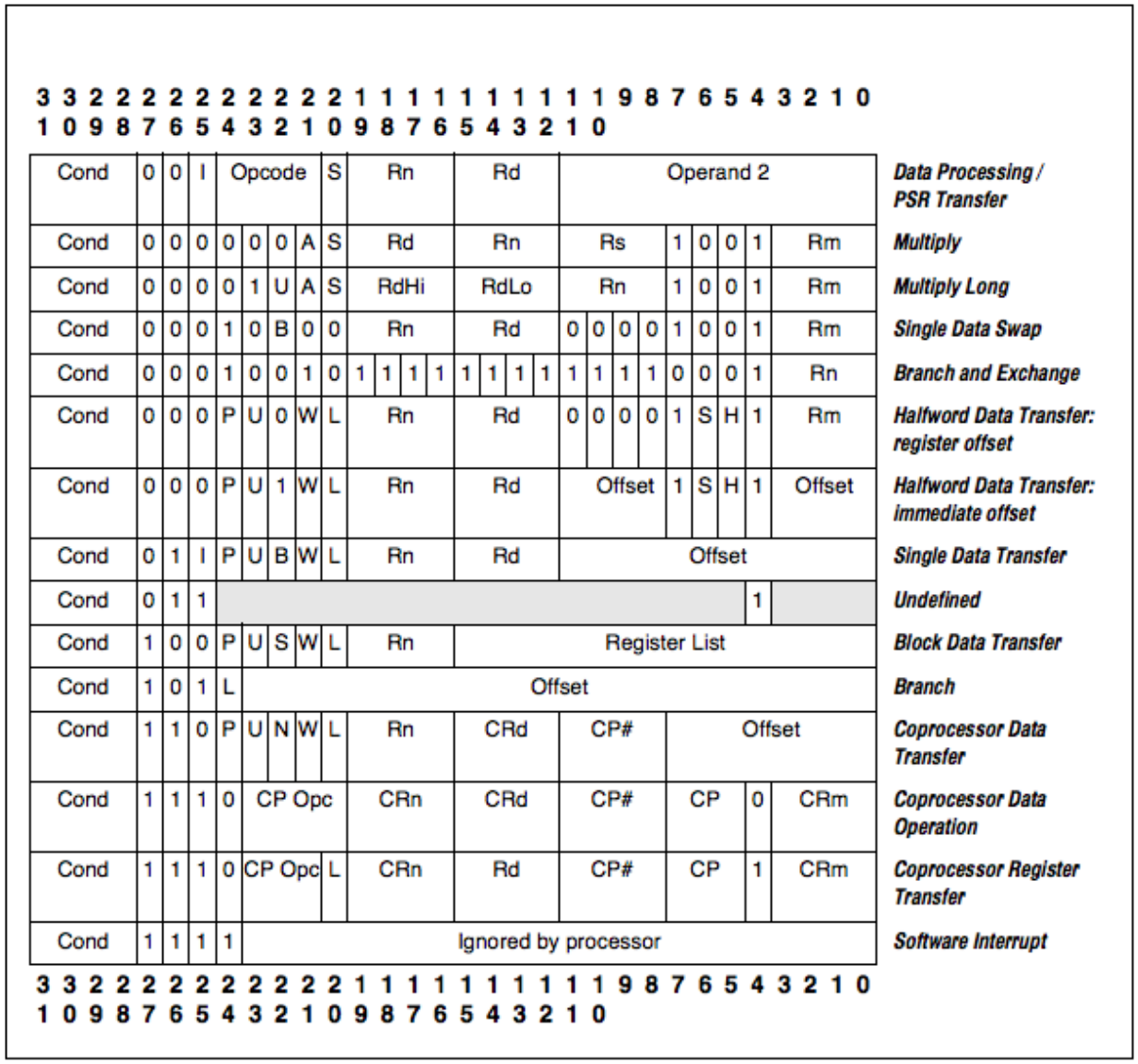
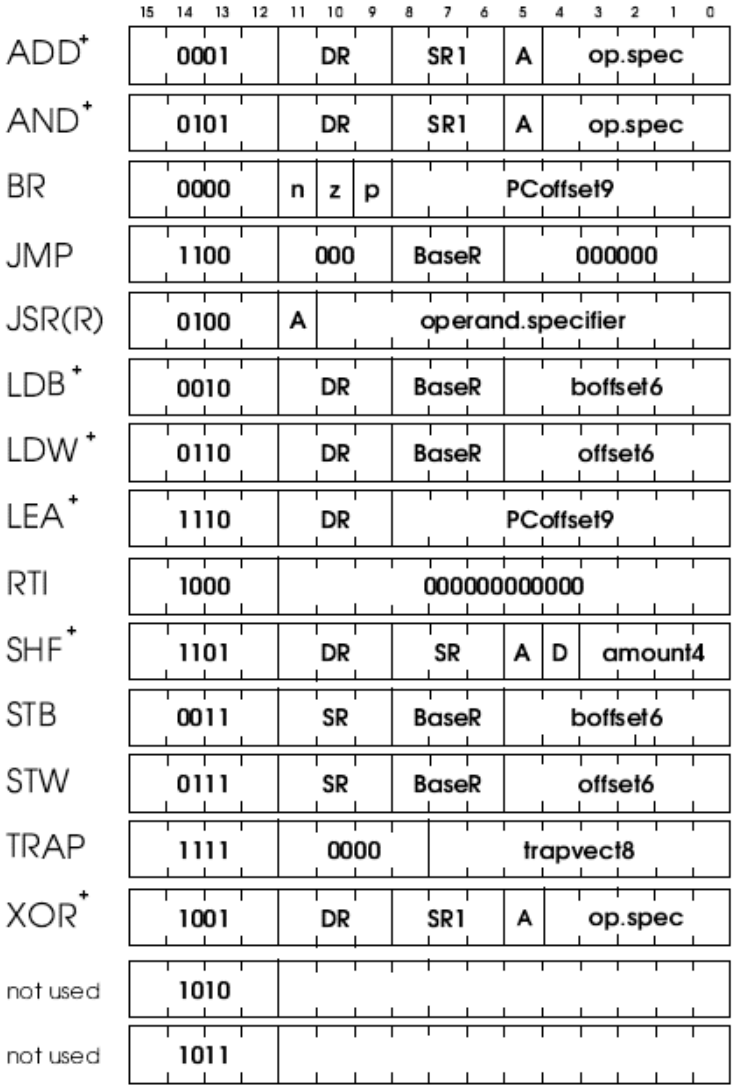


Figure 4-1: ARM instruction set formats

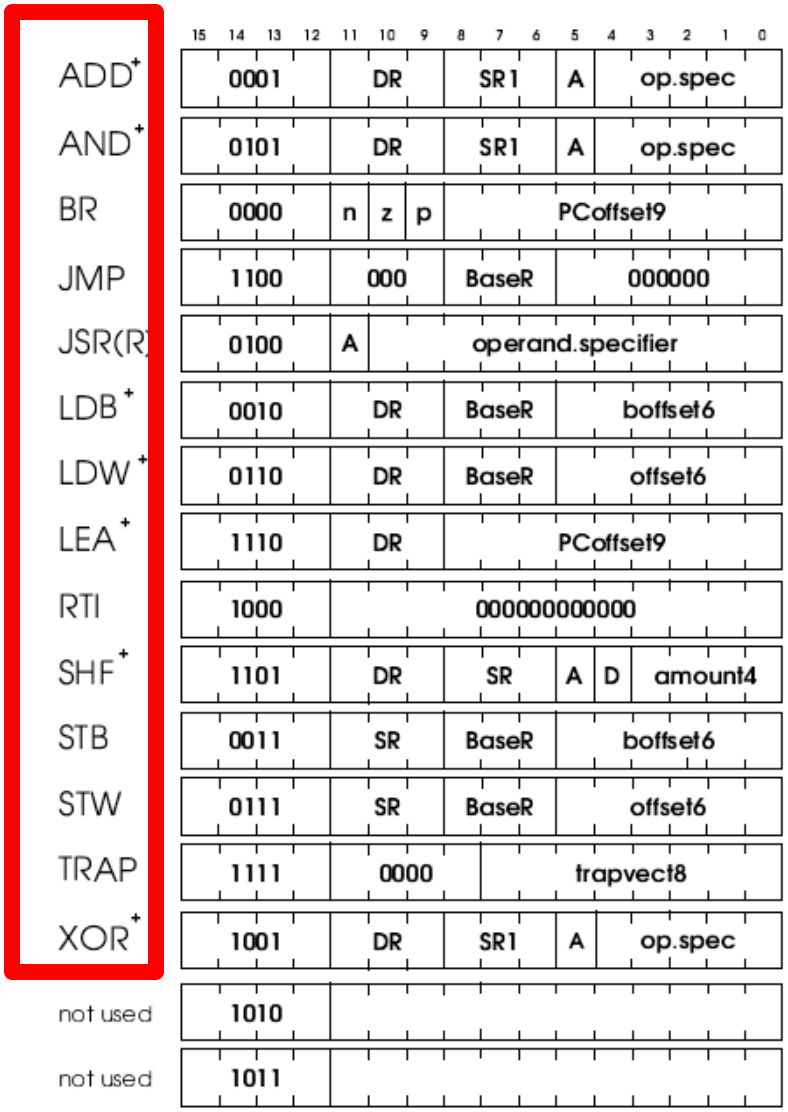
- **条件执行**（Condition Codes）
- **多种寻址模式**
- **协处理器支持**（例如用于浮点或加密指令）
- **块数据传输**（Block Data Transfer），一次可以加载或存储多个寄存器的内容。

指令集、编码与规范



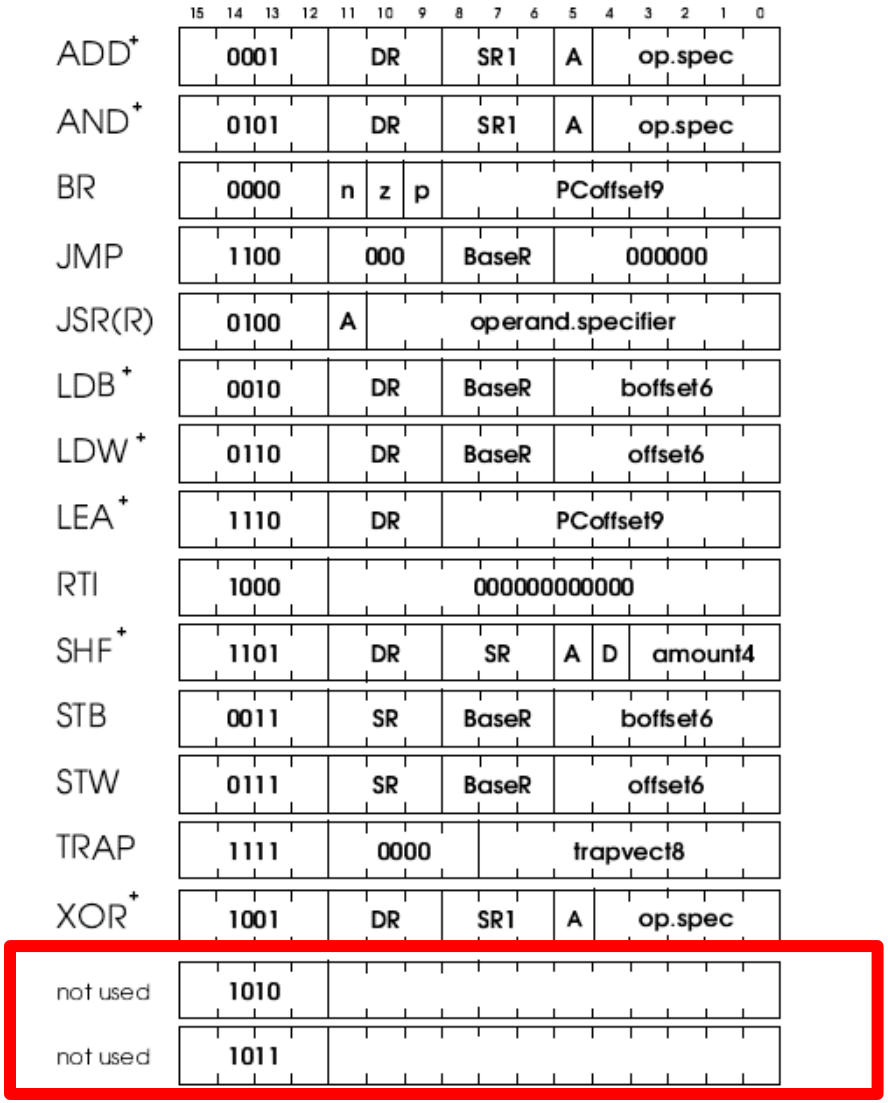
- **LC-3b 指令集（ISA）**
- **x86 手册（x86 Manual）**

指令集、编码与规范



- LC-3b 指令集（ISA）
- x86 手册（x86 Manual）

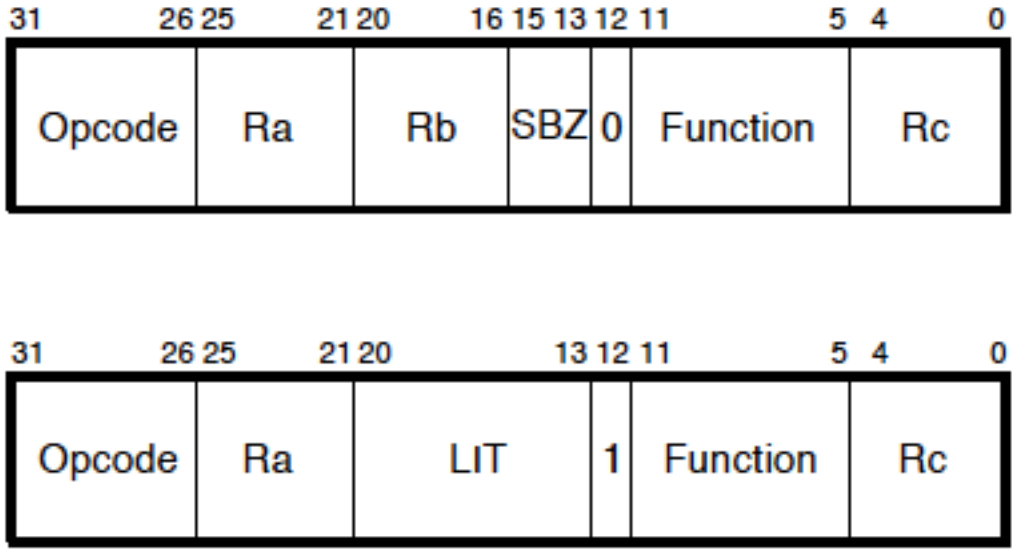
指令集、编码与规范



- LC-3b 指令集（ISA）
- x86 手册（x86 Manual）
- 为什么会存在“未使用的指令”（unused instructions）？➜ 预留空间以便未来扩展或保持编码对齐。

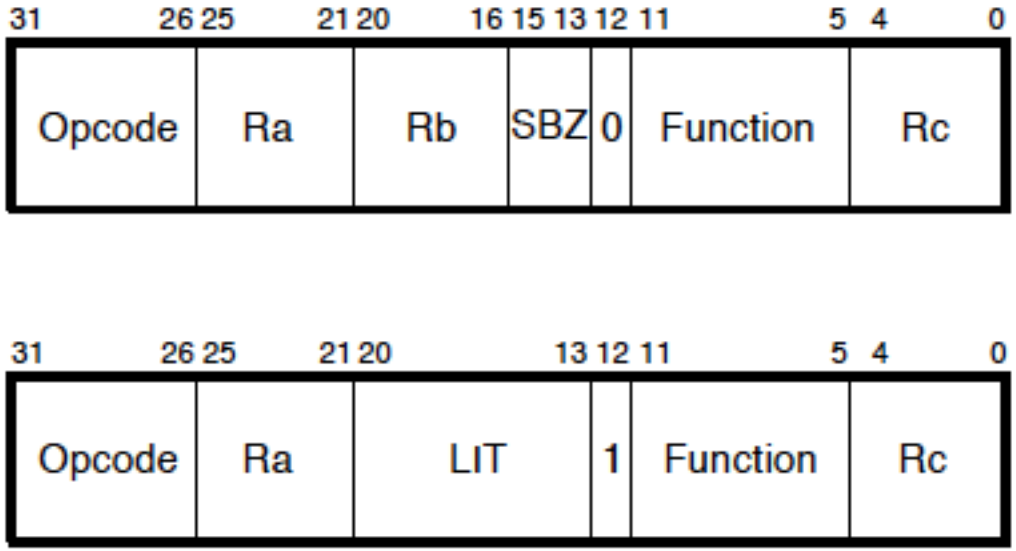
Alpha 架构中的位引导

Figure 3–4: Operate Instruction Format



Alpha 架构中的位引导

Figure 3–4: Operate Instruction Format



- **规则（Bit Steering Rule）**
  - 当指令的第 12 位（bit <12>）= 0 时：→ Rb 字段表示一个源寄存器（Source Register Operand）。
  - 当指令的第 12 位（bit <12>）= 1 时：→ Rb 字段被重新解释为一个 8 位的立即数（literal constant），由位 <20:13> 形成。→ 该立即数会**零扩展（zero-extend）**到 64 位。→ 它表示一个 0 - 255 范围内的正整数。

指令集架构（ISA）的组成元素

- 指令排序模型（Instruction sequencing model）

- 指令处理风格（Instruction processing style）

指令集架构（ISA）的组成元素

- 指令排序模型（Instruction sequencing model）
  - 控制流（Control flow） vs. 数据流（Data flow）
  - 权衡取舍？（Tradeoffs?）
- 指令处理风格（Instruction processing style）

指令集架构（ISA）的组成元素

- **指令排序模型（Instruction sequencing model）**
  - 控制流（Control flow） vs. 数据流（Data flow）
  - 权衡取舍？（Tradeoffs?）

- **指令处理风格（Instruction processing style）**
  - 规定一条指令操作的“操作数”（operands）数量以及其操作方式
  - 0、1、2、3 地址机器：
    - **0 地址机器：** 堆栈机（stack machine）
    - **1 地址机器：** 累加器机（accumulator machine）
      2 地址机器： 双操作数机（2-operand machine）
      3 地址机器： 三操作数机（3-operand machine）

指令集架构（ISA）的组成元素

- **指令排序模型（Instruction sequencing model）**
  - 控制流（Control flow） vs. 数据流（Data flow）
  - 权衡取舍？（Tradeoffs?）

- **指令处理风格（Instruction processing style）**
  - 规定一条指令操作的“操作数”（operands）数量以及其操作方式
  - 0、1、2、3 地址机器：
    - **0 地址机器：** 堆栈机（stack machine）
    - **1 地址机器：** 累加器机（accumulator machine）
      2 地址机器： 双操作数机（2-operand machine）
      3 地址机器： 三操作数机（3-operand machine）

- **权衡取舍（Tradeoffs）**
  - 较大的操作指令 vs. 更多的执行操作
  - 代码大小 vs. 执行时间 vs. 片上内存空间（on-chip memory space）

一个例子：堆栈机（Stack Machine）

- ✅ 优点（+）
  - 指令体积小（操作指令不需要操作数）
    - 逻辑更简单（Simpler logic）
    - 代码更紧凑（Compact code）
  - 过程调用效率高：所有参数都在堆栈中
    - 不需要额外的周期来传递参数（No additional cycles for parameter passing）

一个例子：堆栈机（Stack Machine）

- ✅ 优点（+）
  - 指令体积小（操作指令不需要操作数）
    - 逻辑更简单（Simpler logic）
    - 代码更紧凑（Compact code）
  - 过程调用效率高：所有参数都在堆栈中
    - 不需要额外的周期来传递参数（No additional cycles for parameter passing）
- ❌ 缺点（--）
  - 不容易用“后缀表示法”表达的计算，难以映射到堆栈机上
    - 无法同时对多个值进行操作（一次只能处理堆栈顶部的 N 个值）
    - 灵活性较差（Not flexible）

一个例子：堆栈机（Stack Machine）

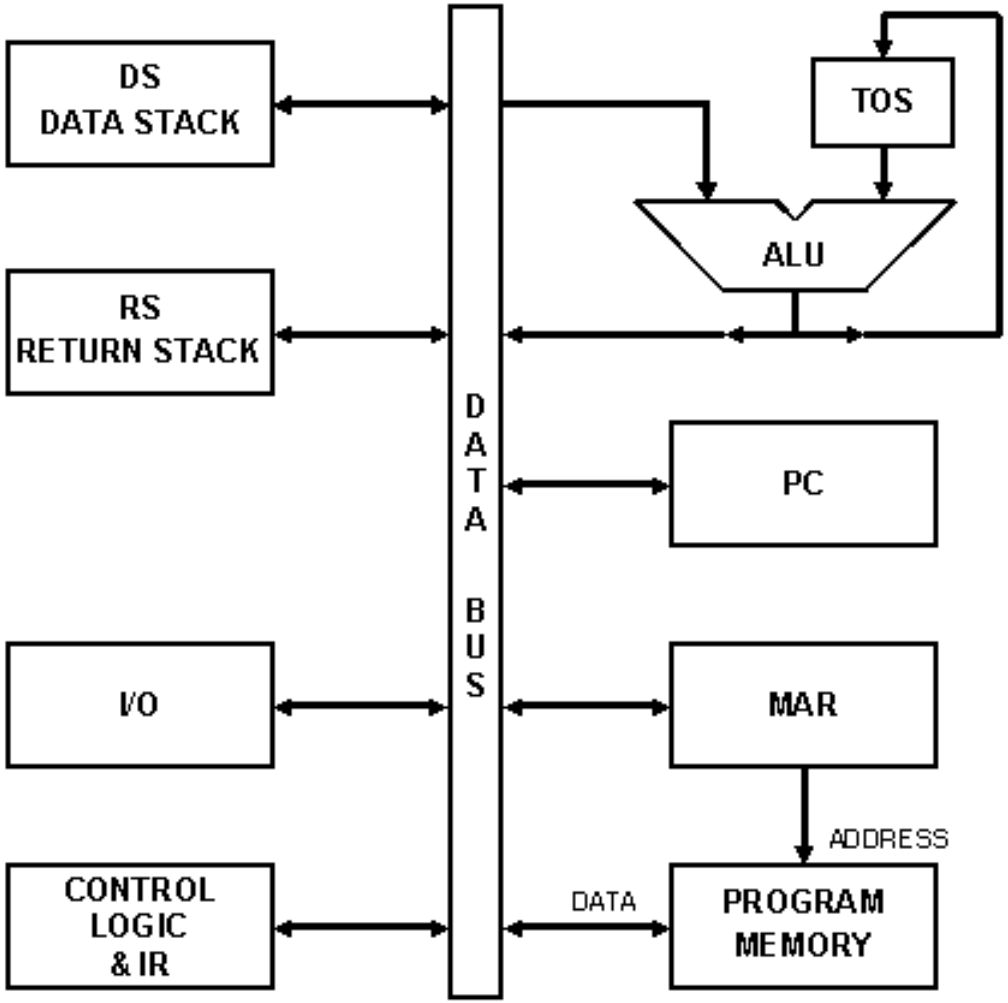


Figure 3.1 -- The canonical stack machine.

Koopman, “Stack Computers: The New Wave,” 1989.

一个例子：堆栈机（Stack Machine）

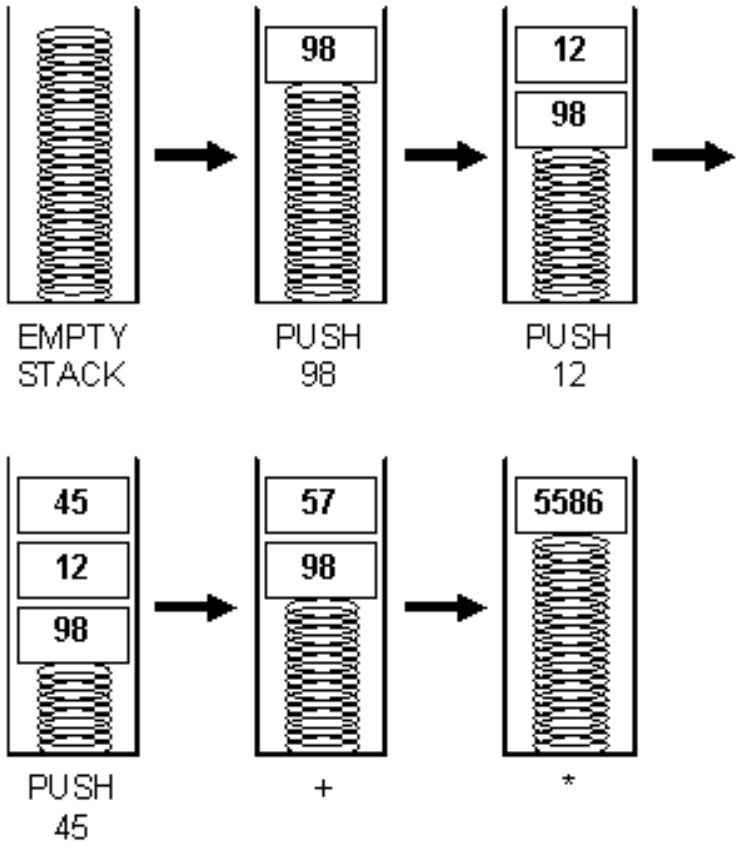


Figure 3.2 -- An example stack machine.

- Koopman, “Stack Computers: The New Wave,” 1989.

指令集架构（ISA）的组成元素

ISA 是软件与处理器之间的约定，规定程序可见的操作与行为

本组聚焦：1. 指令与寻址　2. 数据表示　3. 专门操作支持

1. 指令与寻址：做什么操作，数据从哪里来

操作码规定操作；操作数信息指定数据或位置

a = b + c　　示意汇编：ADD R1, R2, R3

假设 b、c 已在 R2、R3 中，执行后 R1 = R2 + R3

ADD 是助记符；R2、R3 为源，R1 为结果位置

寄存器：处理器内部暂存数据的位置

寻址方式：怎样取得操作数

C 中的例子	取得操作数的方法	本例中的含义
b + 5	立即数	5 直接写入指令
*p	按指针访问	p 保存的是内存地址
A[2]	数组起址 + 偏移	1000 + 2 × 4 = 1008

数组示例：按字节编址，起址 1000，每个元素 4 字节

地址计算与读取可能需要多条指令，具体由 ISA 规定

ISA 的组成元素：数据表示

2. 数据表示：位宽、编码格式及相应运算规则

同一个字节

01000001

解释规则	结果
无符号整数	65
ASCII 字符	A

普通内存通常不会为每个字节自动附加类型标签

同样写成 +，底层规则可以不同

int 加法	整数编码；按整数规则运算
float 加法	浮点格式；考虑指数、有效数字与舍入

十进制编码也由二进制位构成

例：25 的二进制为 00011001；BCD 为 0010 0101

BCD 分别编码各十进制数字；机器编码 ≠ 源码书写进制

ISA 的组成元素：专门操作支持

3. 专门操作：对队列、位域、字符串和栈的支持

数据组织	指令	规定的操作
双向队列	VAX INSQUE / REMQUE	插入 / 移除节点 更新前驱、后继链接
位域	VAX FFS / FFC	在规定范围内 查找第一个为 1 / 0 的位
字符串扫描	x86 SCAS	累加器与一个内存元素比较 更新地址；配合重复前缀连续扫描
栈	x86 PUSH / POP	压栈 / 出栈 并更新栈指针

使用前提：遵守指令规定的数据布局和操作约定

没有专门指令，也可用读取、写入、地址计算等基本指令组合实现

指令集架构（ISA）的组成元素

- **指令（Instructions）**
  - 操作码（Opcode）：规定执行什么操作。
  - 操作数说明信息（Operand specifiers）：指定操作数或其位置。
  - 寻址方式（Addressing modes）：规定如何取得操作数，例如立即数、寄存器以及通过地址访问内存
- **数据类型（Data types）**
  - 定义： 信息的表示形式，用于描述指令所操作的数据表示方式
  - 整数（integer）、浮点数（floating point）、字符（character）、二进制（binary）、十进制（decimal）
  - 双向链表（doubly linked list）、队列（queue）、字符串（string）、位向量（bit vector）、堆栈（stack）
    - **VAX**: INSQUEUE 和 REMQUEUE 指令用于操作双向链表或队列; FINDFIRST
    - **Digital Equipment Corp**: *VAX11 780 Architecture Handbook*, 1977
    - **X86**: SCAN 操作码可对字符串进行操作; PUSH / POP

数据类型的权衡

- 在 ISA 中拥有更多或更高级别的数据类型有什么好处？又有什么缺点？

- 思考角度： 编译器 / 程序员 vs. 微体系结构设计者

数据类型的权衡

- 在 ISA 中拥有更多或更高级别的数据类型有什么好处？又有什么缺点？

- 思考角度： 编译器 / 程序员 vs. 微体系结构设计者

- 语义鸿沟（Semantic Gap）的概念
  - 数据类型与语义层次（或指令复杂度）紧密耦合

数据类型的权衡

- 在 ISA 中拥有更多或更高级别的数据类型有什么好处？又有什么缺点？

- 思考角度： 编译器 / 程序员 vs. 微体系结构设计者

- 语义鸿沟（Semantic Gap）的概念
  - 数据类型与语义层次（或指令复杂度）紧密耦合

- 示例：早期 RISC 架构 vs. Intel 432
  - 早期 RISC： 仅支持整数数据类型
  - Intel 432： 支持对象数据类型，是基于“能力”（capability-based）的机器

指令集架构（ISA）的组成元素

- **内存组织（Memory organization）**
  - 地址空间（Address space）：内存中有多少个可唯一标识的位置。
  - 可寻址性（Addressability）：每个可唯一标识的位置可以存储多少数据。
    - **按字节寻址（Byte addressable）**：大多数 ISA 是按字节寻址，字符通常是 8 位（8 bits）。
    - **按位寻址（Bit addressable）**：例如 Burroughs 1700。为什么？
    - **按 64 位寻址（64-bit addressable）**：某些超级计算机采用此方式。为什么？
    - **按 32 位寻址（32-bit addressable）**：最早的 Alpha 处理器使用此方式。
  - 对虚拟内存的支持（Support for virtual memory）